HOMEWORK 01

→ Objectives

Practice C Programming

→ sumOfDigits(n)

☒ Write the function sumOfDigits

- takes a positive integer n
- returns an int which is the sum of its digits

NOTE: If n is less than or equal to 0, return "-1".

→ UABMaxMinDiff(arr)

☒ Write the function UABMaxMinDiff

- takes an array of integers arr
- return the difference between the maximum and minimum elements in the array.

NOTE: You can pass the size of the array as an input parameter.

→ replaceEvenWithZero(arr)

☒ Write the function replaceEvenWithZero

- takes an array of integers arr
- returns a new array

NOTE: The function should replace every even number in the array with 0. You can pass the size of the array as an input parameter.

→ perfectSquare(n)

☒ Write the function perfectSquare

- takes an integer n
- returns True if n is a perfect square, and False otherwise

NOTE: A perfect square is a number that can be expressed as (k)(k) for some integer k.

→ countVowels(s)

☒ Write the function countVowels

- takes a string s
- returns an integer

NOTE: The function should count the number of vowels (a, e, i, o, u) in the string, ignoring case.

→ Homework Instructions / Grading Rubric

1) Function Implementation:

   ☒ You are required to implement all five functions.

   ☒ Ensure that each function is correctly implemented and passes all test cases.

   ☒ A problem is correct if all our tests pass, otherwise it will be considered as False.

2) File Structure:

   ☒ Single .c File: All your functions must be written in a single .c file.

   ☒ Name the file blazerid_HW01.c

   ☒ Do not submit multiple .c files.

3) Main Function:

   ☒ Single main Functions

   - Your blazerid_HW01.c file should include a single main function that calls each of the five functions with the sample inputs provided in the assignments.

4) General Coding Standards:

   ☒ Follow standard C programming conventions →

   a) including proper indentation
   b) variable naming, and commenting where necessary
   c) ensure your code is readable and free of unnecessary complexity.